Bike Ilha Club

Problema, solução, arquitetura, classes e decisões

Sistema de aluguel de bicicletas de uma locadora de São Luís (MA), com recomendação da bicicleta certa para cada cliente

Documento da equipe | novembro de 2026 | Substitui o diagrama de classes do modelo de várias locadoras

O problema

- A operação de uma locadora de bicicletas (frota, reservas, cobrança, retirada e devolução) fica espalhada e pouco digitalizada, e depende de controle manual.
- O cliente não sabe qual bicicleta serve para o seu tamanho e para o que vai fazer (passeio, trilha, esporte). Escolhe errado, pedala desconfortável e a locadora perde o cliente.
- Sem controle de disponibilidade por período, a mesma bicicleta pode ser prometida duas vezes.
- Sem registro de retirada e devolução (checklist e fotos), danos e furtos são difíceis de comprovar.
- Dados pessoais (CPF, documento) exigem cuidado com a LGPD.

Premissas do README, a confirmar com a locadora.

A solução

- Aplicativo web instalável (PWA) para uma única locadora, com área do cliente e painel da equipe.
- Recomendação: o cliente informa altura, experiência e atividade, e vê as bicicletas livres no período, ordenadas pela adequação, com o motivo de cada uma.
- Reservas que nunca se sobrepõem: a regra é garantida pelo próprio banco de dados.
- Pagamento por Pix e cartão (Mercado Pago), direto para a locadora.
- Retirada e devolução com checklist e fotos; registro de ocorrências.
- Avisos por WhatsApp, e-mail e web push; auditoria de reservas, estoque e valores.

Escopo e status

Escopo (decisão de novembro de 2026, por sugestão do professor):

- Uma única locadora, em São Luís; devolução no mesmo ponto da retirada.
- Fora: várias locadoras, busca da mais próxima, comissão e repasse, assinaturas.

Já existe:

- Projeto Next.js com módulos e CI.
- Cadastro e login de clientes, com limite de tentativas e sessão.
- Banco no Neon com migrations (o modelo ainda é o de várias locadoras).

Falta (6 fases em docs/mudanca-de-escopo.md):

- Simplificar banco e código, catálogo, reservas com recomendação, pagamentos, operação e avisos.

Equipe

Pessoa	Papel	Entregas
Luis Felipe	Tech lead	Banco de dados, migrations, revisão de código
Hélio Levi	Back-end	Aplicação, módulos, contas e conexão com o banco
Renan	Telas	Telas de login e cadastro e gestão da conta do cliente
William	DevOps e QA	Integração contínua e validação do backend

O que há neste documento

1. Problema e solução (esta página)
2. Stack e arquitetura
3. Diagrama de classes do domínio
4. A classe mais importante: Reserva (e a recomendação)
5. Decisões do projeto

Stack e arquitetura

Monólito modular em Next.js, com o banco como guardião das regras

Stack

Camada	Tecnologia	Observação
Aplicação e telas	Next.js 16 (React 19) com TypeScript, PWA	Um só projeto para front-end e back-end; deploy único
Backend	Route Handlers do Next.js (TypeScript)	API em src/app/api; regras em módulos
Banco de dados	PostgreSQL no Neon (extensões: citext, btree_gist; PostGIS opcional)	Constraints, exclusion constraint e segurança por linha
Acesso ao banco	pg (pool), SQL parametrizado e migrations próprias	Sem ORM; runner db/migrar.mjs
Pagamentos	Mercado Pago (Pix e cartão)	Confirmação por webhook; sem repasse
Avisos	Web push (PWA), WhatsApp API e e-mail	WhatsApp é o canal principal (push limitado no iPhone)
Arquivos	Armazenamento privado (Vercel Blob ou S3)	Só a chave fica no banco; URL temporária
Hospedagem	Vercel, com HTTPS	Preview por PR; Production na main
Qualidade	ESLint, TypeScript estrito, node --test, GitHub Actions	CI roda lint, typecheck, testes, migrations e build

Caminho de uma requisição (exemplo: login)

1. A tela envia POST /api/auth/login com e-mail e senha.
2. A rota entrega a requisição ao módulo contas, que lê o corpo (máx. 16 KiB) e o IP.
3. A aplicação confere os limites de tentativas, valida os dados e busca o usuário com SQL parametrizado.
4. A senha é conferida com scrypt, em tempo constante (também quando o e-mail não existe).
5. Dá certo: devolve { data: { token, usuario } }. Dá errado: { error: { code, message } } com o status certo (400, 401, 409, 413, 429).

Arquitetura em camadas

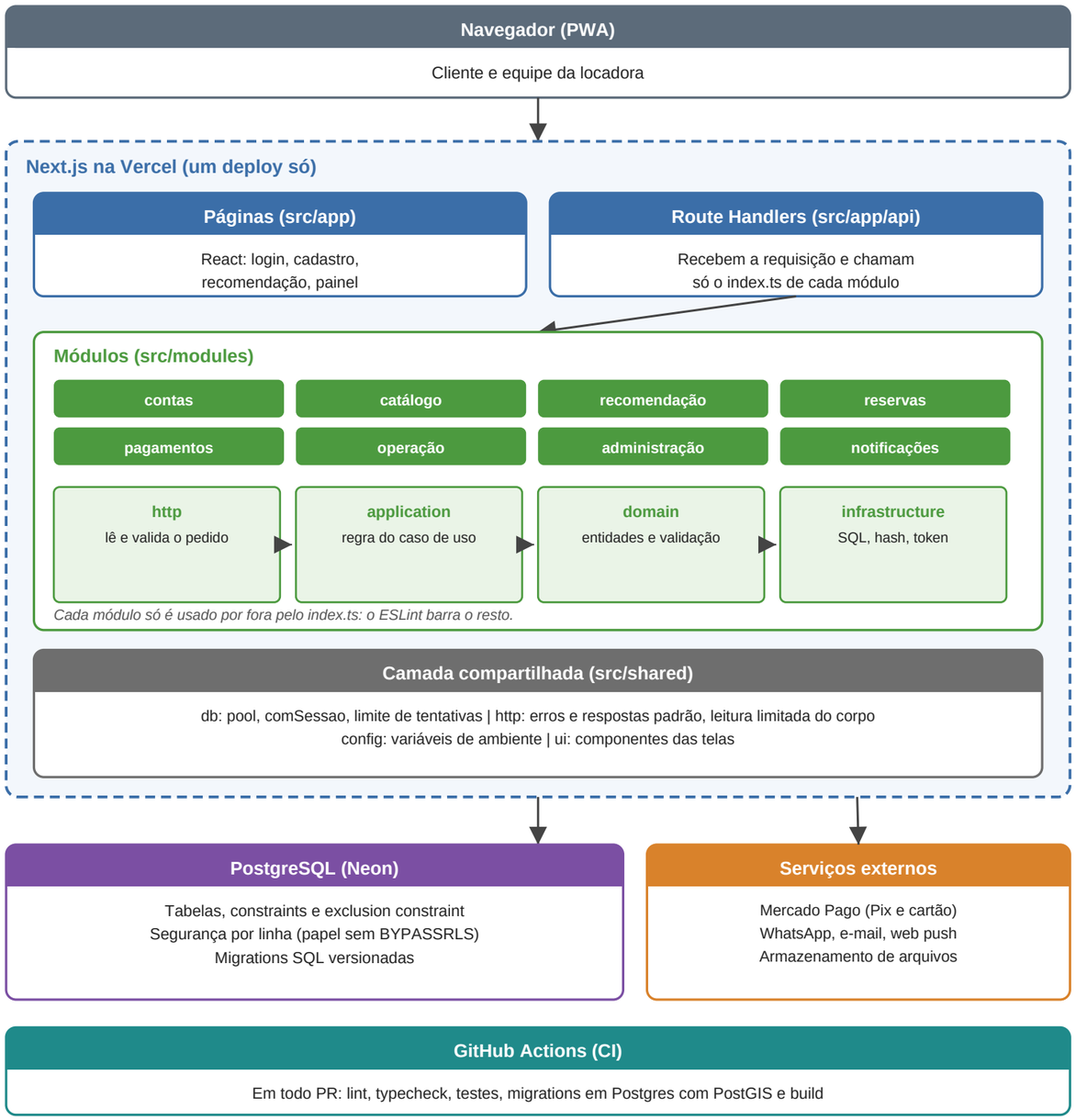
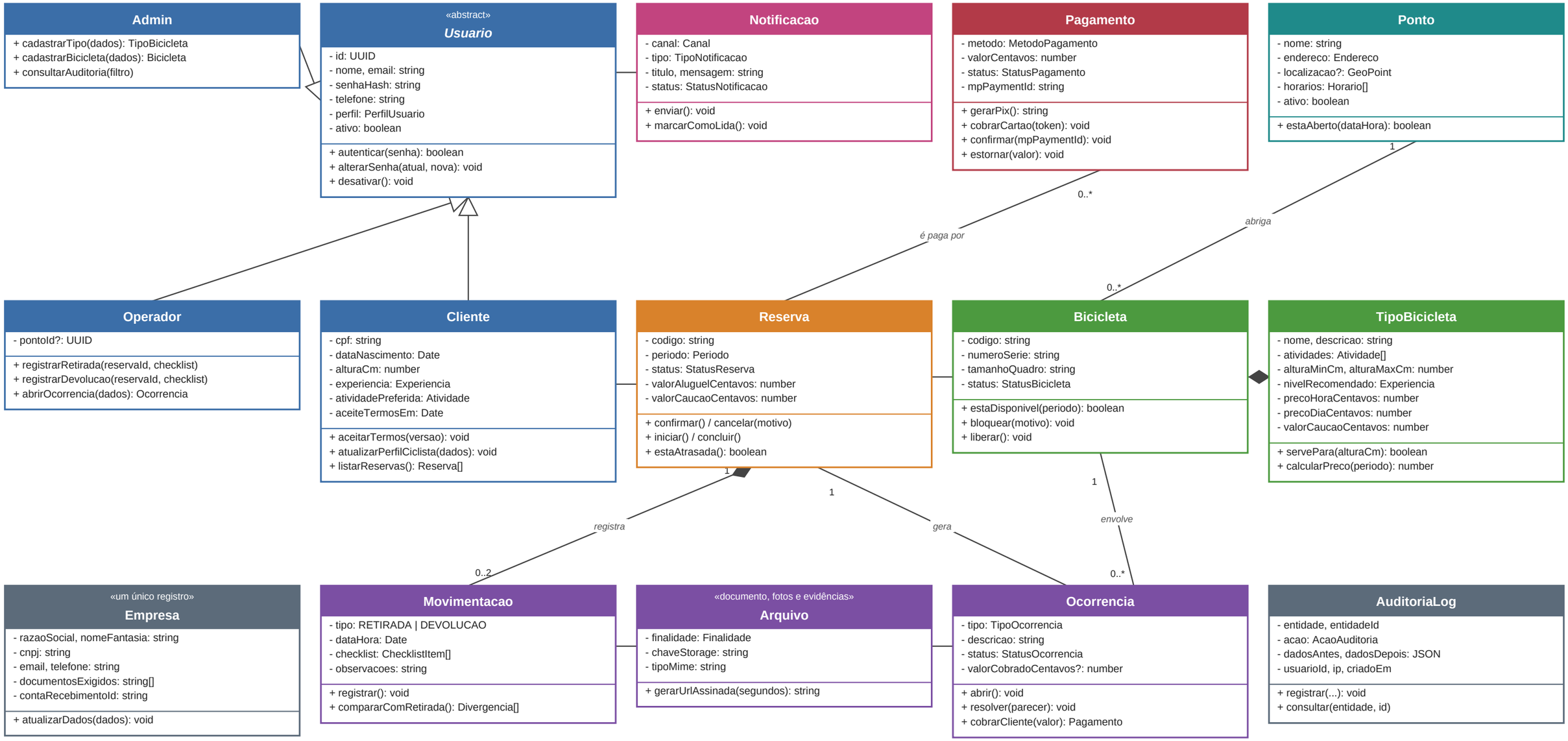


Diagrama de classes do domínio

Modelo para uma única locadora (substitui o das várias locadoras)



Legenda: Herança · Composição (parte-de) · Associação · Cores: azul = contas, verde = catálogo, laranja = reservas, vermelho = pagamentos, roxo = operação, rosa = avisos, cinza = administração.

Serviços de aplicação (RecomendacaoService, ReservaService, PagamentoService) estão na página 4. Valores em dinheiro são centavos inteiros; o operador atua num Ponto (ponto_id opcional).

A classe mais importante: Reserva

É o centro do domínio e carrega a regra que mais importa: nunca duas reservas para a mesma bicicleta

Por que a Reserva? Ela liga o cliente, a bicicleta, o ponto, o pagamento, a retirada e a devolução e as ocorrências. É o que o cliente compra, o que a locadora opera e o que o dinheiro move. Quase todo requisito do README passa por ela, e é nela que o banco garante a regra de negócio mais cara de errar: a mesma bicicleta prometida a duas pessoas.

Reserva

- id: UUID
- codigo: string (único, mostrado ao cliente)
- clienteId: UUID
- bicicletaId: UUID
- pontoId: UUID
- periodo: Periodo (início e fim, tstzrange)
- status: StatusReserva
- valorAluguelCentavos: number
- valorCaucaoCentavos: number
- criadaEm: Date
- canceladaEm?: Date

+ confirmar(): void
+ cancelar(motivo): void
+ iniciar(): void
+ concluir(): void
+ estender(novoFim): void
+ calcularValor(): number
+ estaAtrasada(): boolean

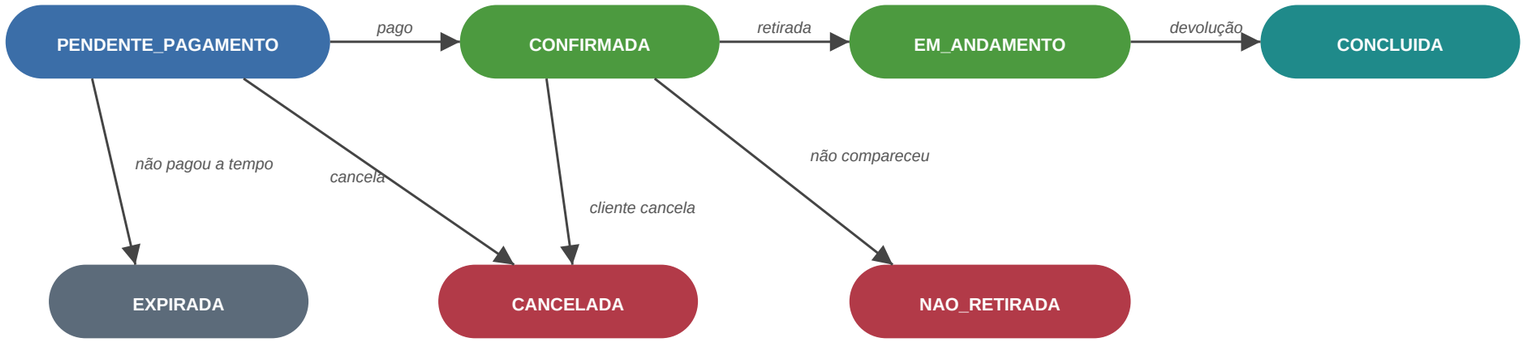
Regras que a Reserva garante

- Duas reservas ativas da mesma bicicleta não podem se sobrepor (exclusion constraint no banco).
- Só reserva bicicleta com status DISPONIVEL.
- Retirada e devolução no mesmo ponto.
- A reserva não paga expira e libera a bicicleta.
- Cada cliente enxerga só as próprias reservas (segurança por linha).
- Mudanças de status, de período e de valor vão para a auditoria.

Com quem a Reserva se relaciona

Classe	Tipo	Papel
Cliente	N para 1	quem faz a reserva
Bicicleta	N para 1	o que é reservado
Ponto	N para 1	retirada e devolução
Pagamento	1 para N	Pix ou cartão
Movimentação	1 para até 2	retirada e devolução
Ocorrência	1 para N	dano, atraso, furto

Ciclo de vida (StatusReserva)



A reserva pendente segura a bicicleta (conta na exclusion constraint) até o pagamento ser aprovado ou o prazo vencer. Expirada, cancelada e não retirada liberam a bicicleta.

A regra no banco de dados

```
ALTER TABLE reserva
  ADD CONSTRAINT reserva_sem_sobreposicao
  EXCLUDE USING gist (bicicleta_id WITH =, periodo WITH &&)
  WHERE (status IN ('PENDENTE_PAGAMENTO', 'CONFIRMADA', 'EM_ANDAMENTO'));
```

Funciona mesmo com dois cliques simultâneos: o banco recusa o segundo. A verificação no código só serve para responder rápido ao cliente.

O diferencial do produto: RecomendacaoService

«service»
RecomendacaoService

- pesoAltura = 50
- pesoAtividade = 30
- pesoExperiencia = 20

+ recomendar(cliente, periodo, pontoId): ItemRecomendado[]
+ pontuar(tipo, cliente): number
+ motivo(tipo, cliente): string

Como a recomendação decide

- Filtra: tipos ativos com unidade livre no período e faixa de altura que serve ao cliente.
- Pontua de 0 a 100: altura 50 (melhor no centro da faixa), atividade 30, experiência 20.
- Desempata pelo menor preço por dia e mostra o motivo de cada sugestão.
- Pesos são um ponto de partida, a calibrar com a locadora.

ReservaService

criarReserva(), verificarConflito()
cancelarReserva(), expirarPendentes()

PagamentoService

iniciarPagamento()
processarWebhook(evento)

Decisões do projeto

O que foi decidido, por quê, e o que ficou de fora

Decisão	Por quê	Alternativa deixada de lado
Uma única locadora	Reduzir tamanho e complexidade, por sugestão do professor (novembro de 2026). O núcleo, a recomendação, continua.	Plataforma com várias locadoras: busca da mais próxima, comissão, repasse e isolamento entre locadoras
Monólito modular em Next.js	Um só deploy, equipe pequena e fronteiras claras: o ESLint só deixa um módulo ser usado pelo index.ts.	Microsserviços (custo de operação sem benefício agora); podem sair módulos no futuro
PostgreSQL gerenciado (Neon)	As regras que não podem falhar ficam no banco: constraints, exclusion constraint e segurança por linha.	Banco sem esquema; ORM (o SQL é parametrizado e fica visível e revisável)
Reservas sem sobreposição por exclusion constraint	Garante a regra mesmo com requisições simultâneas, e usa a extensão btree_gist.	Conferir só no código (sujeito a corrida entre duas requisições)
Recomendação por regras explicáveis	Poucos dados no começo, e dá para mostrar o motivo ao cliente e calibrar os pesos com a locadora.	Aprendizado de máquina (sem volume de dados nem necessidade agora)
Dados mínimos do cliente	Só altura, experiência e atividade entram na recomendação (LGPD: minimização). Documento fica em armazenamento privado.	Peso, histórico detalhado e outros dados pessoais
Autenticação própria e simples	Senha com scrypt, token HS256 de 24 h, limite de tentativas, e exigirSessao confere se a conta continua ativa.	Serviço externo de login (dependência e custo); token que nunca expira
Segurança por linha por cliente	Cada cliente só vê os próprios dados; o app conecta com um papel sem BYPASSRLS. Protege contra um filtro esquecido no código.	Confiar só nos filtros das consultas. Limite: a RLS não protege contra injeção de SQL
Dinheiro em centavos, ids UUID, datas em UTC	Evita erro de arredondamento, ids previsíveis e confusão de fuso horário.	Valores decimais em ponto flutuante; ids sequenciais
Migrations SQL versionadas com runner próprio	Cada mudança do banco é um arquivo numerado, com subida e descida, aplicado em transação. Migration já aplicada nunca é editada: cria-se outra.	Alterar o banco à mão no console do Neon
Mercado Pago (Pix e cartão)	Meio de pagamento comum no Brasil, com confirmação por webhook. Sem repasse, pois a locadora é uma só.	Assinaturas recorrentes (ficam para uma fase posterior)
PostGIS opcional	Sem busca por locadora mais próxima, ele só serve para mostrar o ponto no mapa e a distância.	Exigir PostGIS desde o começo
Qualidade automatizada	CI em todo PR (lint, typecheck, testes, migrations em Postgres e build) e testes com node --test, sem dependência extra.	Revisão só manual